

Localisation

The Gettext Framework

Having looked at Indian language localisation in the previous article of this series, let us now look at a major system for internationalisation called Gettext.

Part—3

Even if localisers work with Web-based environments, it is important that they have a good understanding of the underlying details to be able to appreciate the localisation message source string formats. Despite several initiatives to standardise internationalisation support across computer operating systems, it has not been achieved. If one needs to deliver software to run on multiple systems, a software-specific framework is used, and language packs are built specially for such software. The main examples of this kind of software are Firefox and LibreOffice. Let's explore locale briefly before getting into the details of Gettext system, the most common framework with support for various programming languages.

Locale

Locale is the term for the language and country specific practices for reporting date, time, currency etc of the user. A program written in English without any notion of localisation is designated to have the locale 'C', probably arising from the most popular system programming language 'C'. All software supporting localisation needs to operate as per the locale of the user. The locale basically consists of a two-letter language code and a two-letter country code, separated by an underscore (LL_CC). For example, it is *en_US* for US English and *te_IN* for the Indian language Telugu. When software with localisation support executes, it uses the locale definition in the environment to pick the relevant language message catalogs.

The Gettext system

Gettext was originally developed by Sun Microsystems in the early 1990s. GNU released its version in 1995. It is implemented for most programming languages and scripts. This includes a set of tools, guidelines, and a run-time library to utilise translations, plus a few independent programs to manipulate the various messages.

Any computer program has statements defining data, reading data and writing data, apart from computation on the same. An internationalisation system should be able to operate on the data without affecting the rest of the program, which means that the data definition and usage need to pass through special functions. At the same time, the use

of special functions should not distract from the readability of the program. Gettext is the name of the function that is used for this purpose. A function whose name is a simple underscore (`_()`) is used as an alias for *gettext()*, to improve readability. A programmer has to internationalise a program, and then use the usual compilation path for building the program. Another path is required to extract translatable messages, localise and build the localised message set, as well as to place the same in the specified location. More extensive information on the guidelines for writing messages is available in the GNU Gettext manual. These deal with constant strings, plurals and information specifically relevant to the programmer, translator and maintainer.

A primer on internationalisation and localisation

Let's take the simple 'C' language program 'Hello World', as in Listing 1, and look at how it can be internationalised and then localised. Note that the code has been tried out on Ubuntu 12.04, set up with English and Telugu locales, and a proper build environment for the GNU C compiler (GCC 4.6.3). Please ensure the same or a similar working environment before trying these programs.

Listing 1: *hello.c*

```
/*hello_English program*/
#include <stdio.h>
#include <stdlib.h>
int main() {
    printf("Hello World\n");
    return 0;
}
```

Internationalisation

Listing 2: *hello18n.c*

```
/*hello_internationalised program*/
#include <stdio.h>
#include <stdlib.h>
#include <libintl.h>
#include <locale.h>
```